



HPC in Java

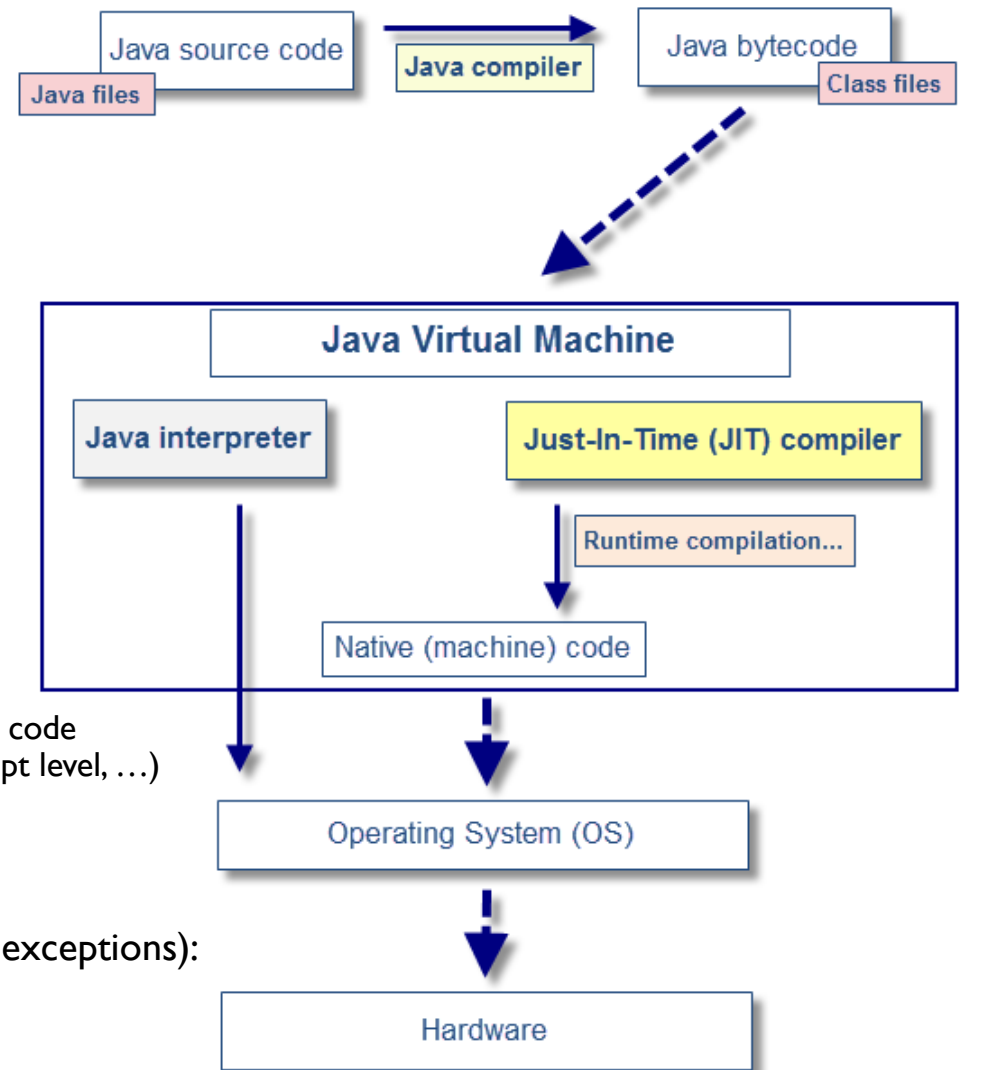
Parallel Algorithms

João Luís Sobral (jls@di ...)

May/2026

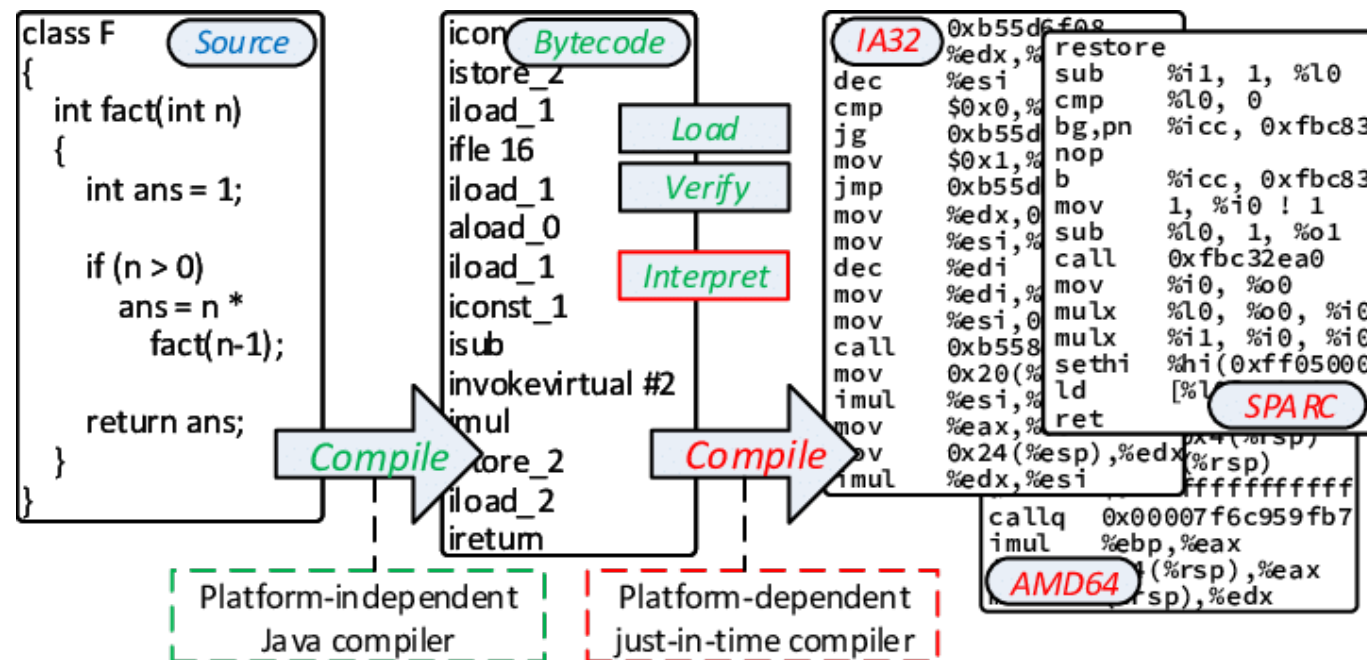
Java compilation & execution (I)

- **Java compiler**
 - Generates "java bytecodes"
 - Bytecodes require a JVM to run
- **Java Virtual Machine**
 - Runs Java bytecodes
 - Always interprets first
 - Includes runtime profiling
 - Hotspot JIT compilation
 - Also includes profiling ...
 - Multiple versions of compiled code (e.g., w/profiling, per thread, opt level, ...)
 - Provides other services
 - Memory management (GC)
 - Safe pointers (raises runtime exceptions):
 - Type safe, boundary checks, null-pointer detection ...



Java compilation & execution (2)

- Bytecode is a "virtual" instruction set
 - Platform-independent & compact (stack machine)

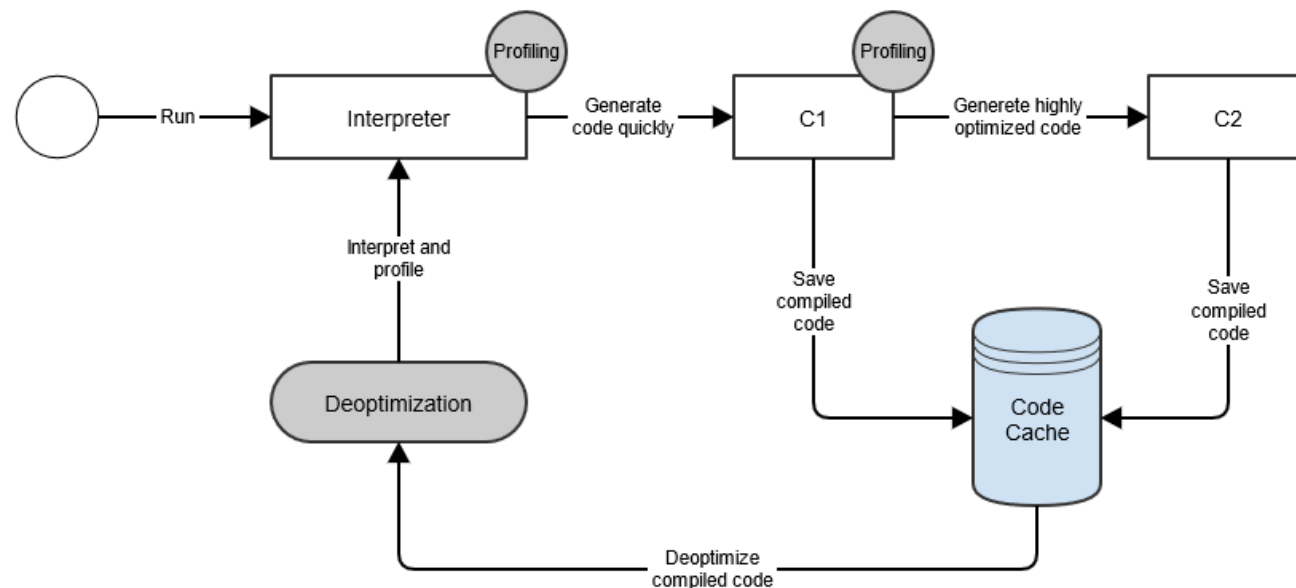


Java compilation & execution (3)

- Main advantages
 - Compilation based on runtime data
 - Execution profile detects “true” hotspots
 - More precise than in compile time
 - GC can perform data layout improvements
 - Mostly performed during memory compaction
 - Note: quite sophisticated memory management...
- Main overheads
 - JIT compile time (clock cycles) and delay
 - Several safety features, array of pointers
 - Heavy used of heap allocated objects
 - Ameliorated with escape analysis, thread-local heap, ...

Java compilation in detail (I)

- Progressively use more aggressive compilation modes
 - Starts in **interpreted mode** with profiling to find hotspots
 - C1 mode provides **fast compilation** (initially with profiling)
 - C2 mode performs **stronger optimizations** (no profiling)
 - De-optimisation: discard optimized versions (uncommon path, ..)

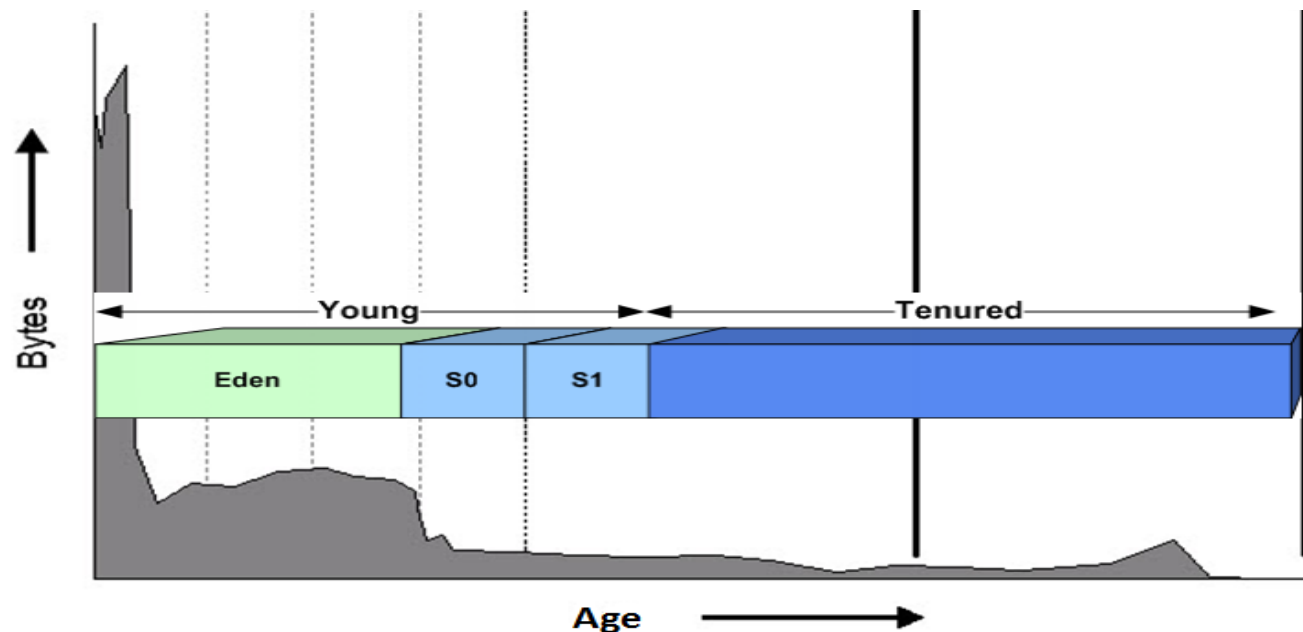


Java compilation in detail (2)

- Five compilation levels:
 - Level 0 - interpreted mode
 - Level 1 – Simple C1 Compiled Code (simplest, no profile)
 - Level 2 – Limited C1 Compiled Code (when c2 queue is full)
 - Level 3 – Full C1 Compiled Code (more common C1 level w/profile)
 - Level 4 – C2 Compiled Code
- Compilation logs
 - `-XX:+PrintCompilation:`
 - **Timestamp** – In milliseconds since application start-up
 - **Compile ID** – Incremental ID for each compiled method
 - **Attributes** – The state of the compilation with possible values:
 - % – On-stack replacement occurred
 - s – The method is synchronized
 - ! – The method contains an exception handler
 - b – Compilation occurred in blocking mode
 - n – Compilation transformed a wrapper to a native method
 - **Compilation level** – Between 0 and 4
 - **Method name**
 - **Bytecode size**
 - **Deoptimisation indicator**

GC

- Memory management
- Runs when the heap is full
- Many implementations of garbage collector
 - Those divide memory regions in generations



Java profiling

- Java introduces many complexities
 - Debug information in binary code (JIT in the middle)
 - Symbol names, stack reconstruction (in sampling-based)
 - Interpreted methods & many binary code versions
 - Safepoints (on native-Java profilers like VisualVM)

Profiler	VisualVM	Perf + perf-map-agent	Async-profiler
Profiling Method	Sampling and Instrumentation	Sampling and Instrumentation	Sampling and Instrumentation
Named interpreted methods	Yes	No	Yes
Inlined methods	If they contain safepoints	With +DebugNonSafepoints flag	With +DebugNonSafepoints flag
JVM code	No	Yes	Yes
Native and kernel functions	No	Yes	Yes
Hardware Performance Counters	No	Yes	Some
Safepoint bias	Yes	No	No
Others	-	Requires +PreserveFramePointer	-

Table 1.: Summary of different Java profiling tools



HP Matrix Multiplication in Java

Parallel Algorithms

João Luís Sobral (jls@di ...)

May/2026

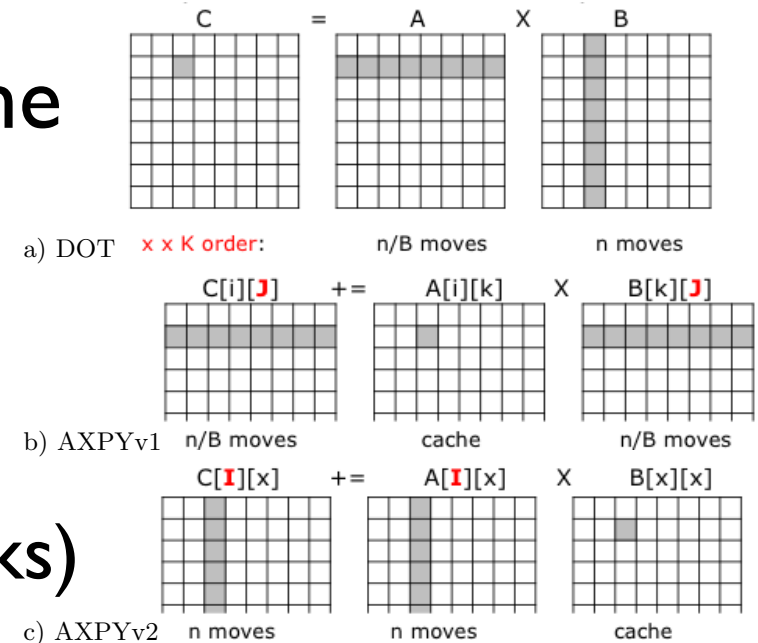
Note: results on Java v15 and Intel Xeon v2, I2c

HPC in Java

- Potential performance issues (#1):
 - Auto-Vectorization
 - Range and null pointer checks
 - Function inlining
 - Loop unrolling
 - Register scheduling
 - JIT overhead & delay
 - Library calls (including thread-related)
- Memory impact
 - Garbage collector
 - Matrix representation as an Array of Pointers
 - Object headers

Recap: approaches for a MM kernel

- Which alternative is the best for a JIT-system?
 - AxPy better than Dot
 - Mandatory AoP layout
 - Costly transpose (checks)
 - Base ijk kernel



```
for (int i=0; i<NR; i++)
  for(int k=0; k<NK; k++) {
    Aik = A[i][k]; // range check A[i][k], row C[i] and row B[k]
    for(int j=0; j<NC; j++) // vectorized loop (AXPY)
      C[i][j] += Aik x B[k][j];
  }
```



Use wide rows to reduce checks

Matrix Multiplication in Java

- Base kernel (Base 2x4 - tiling for registers)

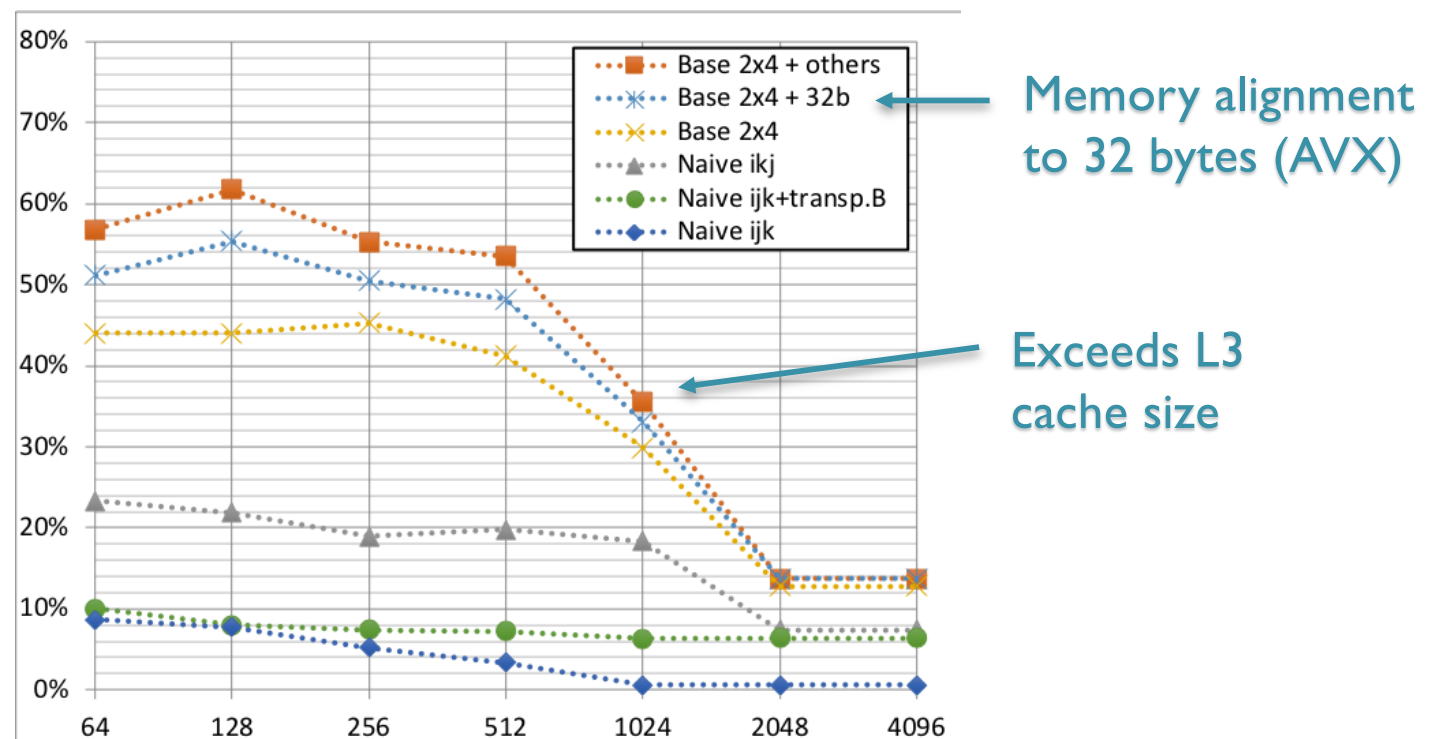


Fig. 6. Performance of several MM kernels in Java in percentage of peak performance (matrix size in the X-axis).

Matrix Multiplication in Java

- Tiling for L1 cache , L2 and L3

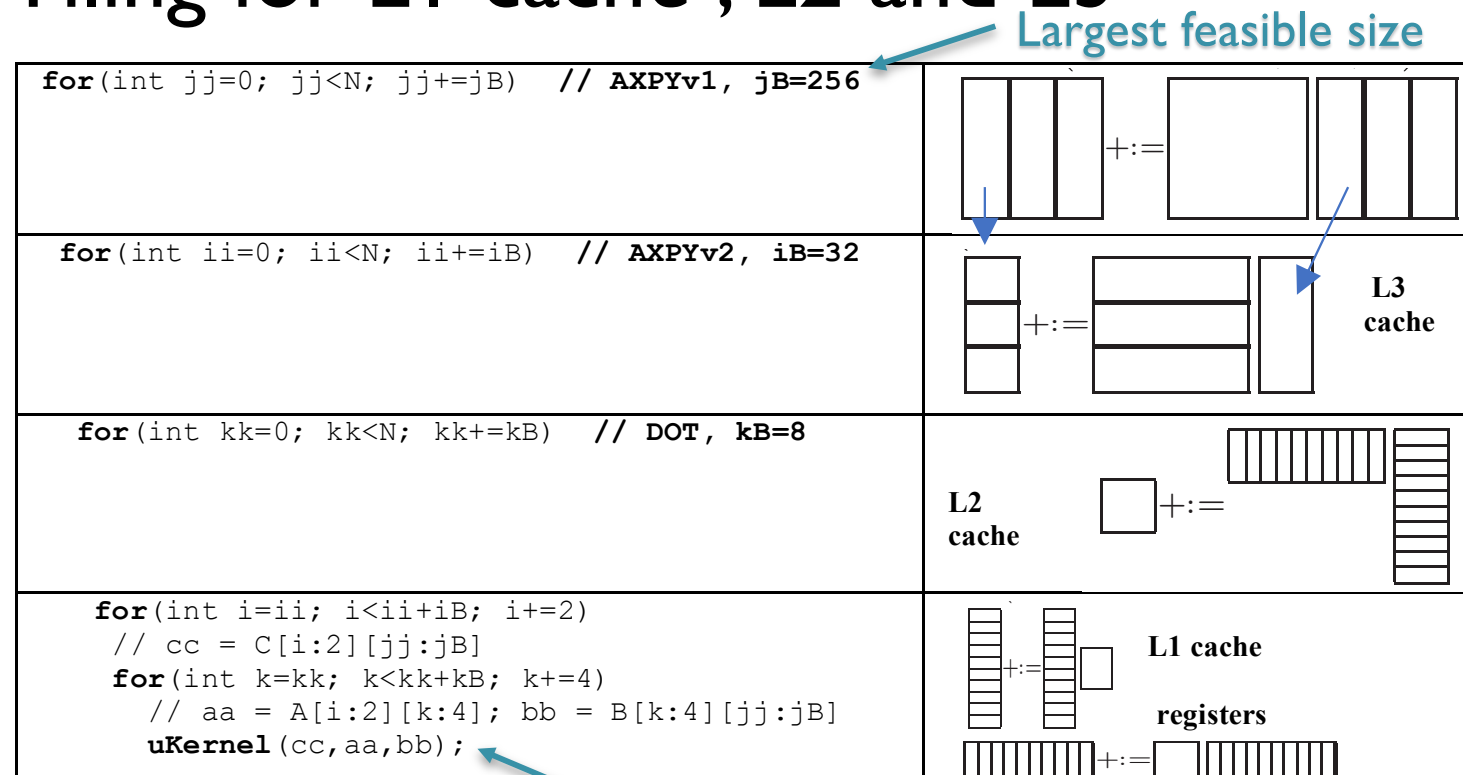


Fig. 10. AXPYv1+AXPYv2+DOT block composition applied to the 2x4 kernel

uKernel is expanded inline

Matrix Multiplication in Java

- Performance of Tiling for L1, L2 and L3

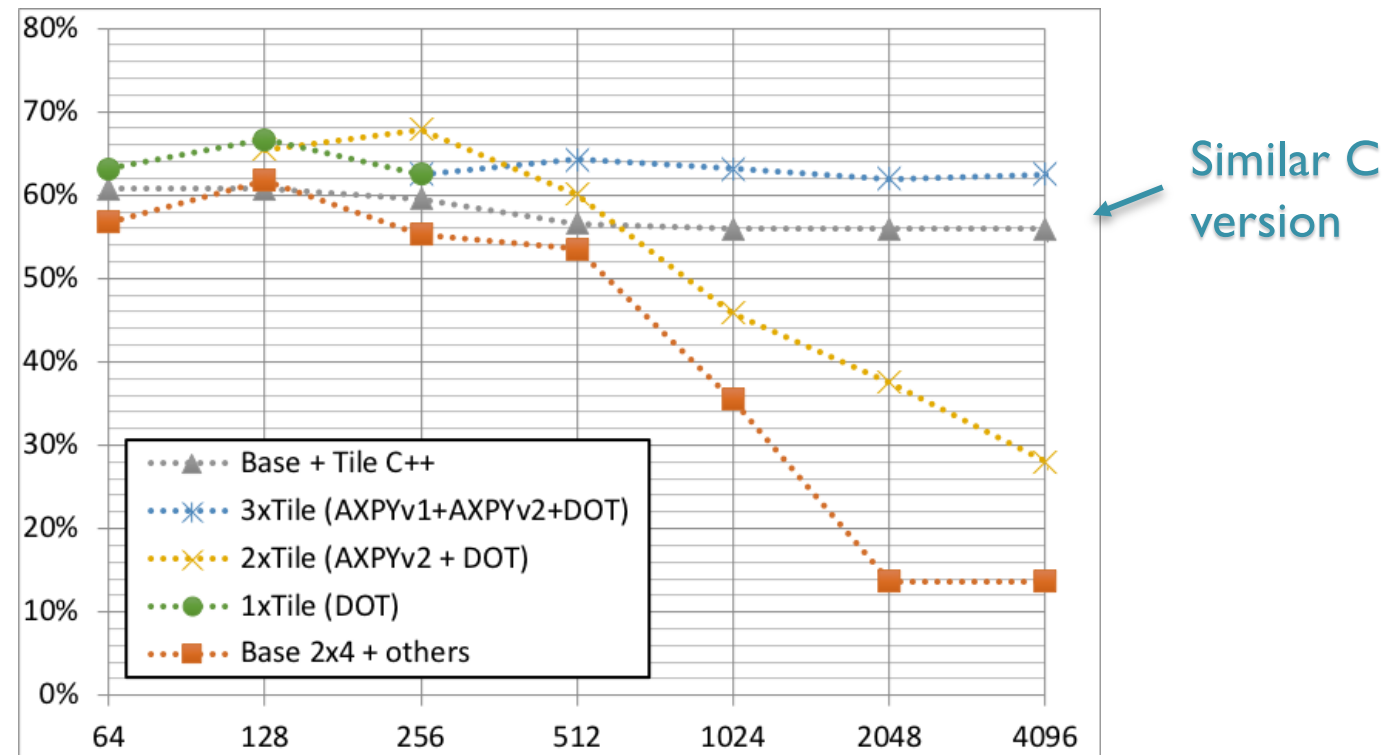


Fig. 11. Performance of several optimised MM kernels in percentage of peak performance (matrix size in the X-axis)

Matrix Multiplication in Java

- Parallel implementation

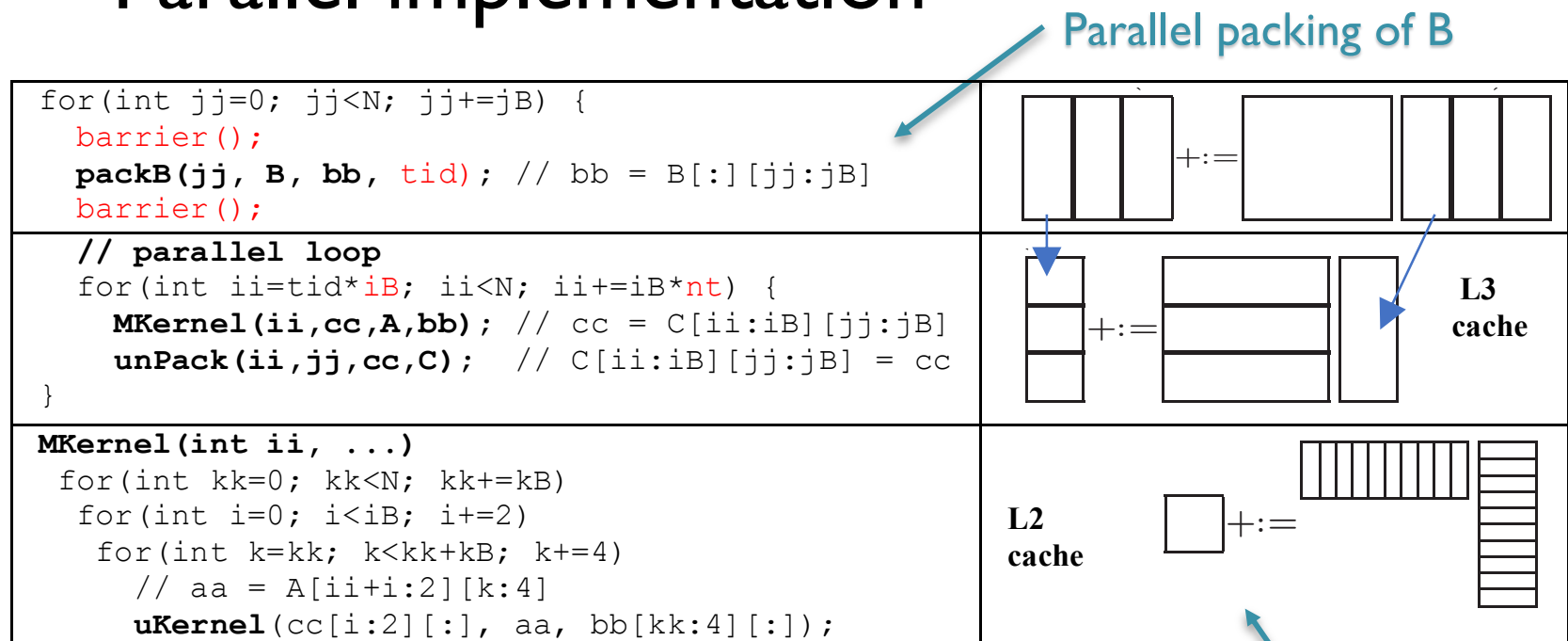


Fig. 12. Basic parallel implementation

Each thread works on a tile of C,
all share the same pack of B

Matrix Multiplication in Java

- Parallel implementation - results

